

Exercise 2: Association, Aggregation & Composition

Pet Clinic Management System

Overview

- **Take-home exercise:** to be completed **outside of class**.
- Deadline: **Tuesday, 26th May 2026**, 23:59 MYT (UTC+8).
- Strategy: **Pair Programming** - two students collaborate online using VS Code Live Share.
- Submission: Submit the source code (**.cpp file**) and the **video link** via elearning before the deadline.

Pair Programming Rules

The Two Roles

 Driver	Types the code and controls the keyboard. Focus on writing clean, working code based on what you and your Navigator have discussed.
 Navigator	Reviews the code in real time, spots errors, suggests improvements, and guides the overall direction of the solution. The Navigator does not type.

Ground Rules

- Decide who will be the **host**. The host opens a Live Share session in VS Code with Read and Write permissions.
- Share the Live Share link with your partner. Use **Google Meet, Webex, or any online meeting tool** to communicate.
- Switch roles at least once. Both members must experience both the Driver and Navigator roles. You may switch after completing any task.
- Stay engaged throughout. The Navigator should speak up actively. If the Navigator is silent, the lecturer will take note.
- Record the session. See the **Video Recording** section for instructions.

Pair Programming Evidence

Since this is a take-home exercise, pair programming is assessed through **two sources of evidence**: the **video recording** of your session and the **Role Declaration comment** in your source code. There is no online log to fill in. Both sources must be consistent with each other.

Part 1: Video Recording

Record your entire pair programming session from start to finish. The video is the primary evidence that you worked together genuinely.

What the video must show:

- Both members **actively talking and discussing** throughout the session; not just one person typing in silence.
- The **Driver must explain out loud** what they are writing and why, for every task. For example: *“I am using composition here because the Vaccine object cannot exist without PetRecord”*. This shows genuine understanding of the concept.
- The **Navigator must actively respond**; pointing out errors, asking questions, or suggesting improvements. The Navigator must not sit silently.
- At least **one role switch** must be clearly visible. When switching, the new Driver must **say out loud**: *“I am now the Driver”* or *“I am now the Navigator”*.
- Your **VS Code editor and console output** must be visible at all times. Increase the font size by pressing **Ctrl + +** in VS Code so your code is easy to read.

How to record:

- Use **Google Meet ‘Record meeting’**, OBS, or any screen recorder. Use your UTM student account on Google Meet to access the recording feature.
- You may record in one or multiple videos. If multiple, put them all in one **Google Drive folder** and share the folder link.
- Upload to **Google Drive**. Set sharing to **Restricted** and add **lizawati@utm.my**. Ensure access is available until end of semester.
- Submit the raw video. No post-editing is required.

⚠ Important - The Use of AI Tools:

You are permitted to use AI tools such as ChatGPT or GitHub Copilot **as a reference only**, in the same way you would use a textbook or lecture note. However, **submitting code that is largely generated by AI without genuine understanding is considered academic misconduct**. The purpose of this exercise is to develop your own understanding of OOP concepts. Submitting AI-generated code defeats this purpose entirely.

The video is the most reliable evidence of genuine understanding. **If your video shows you cannot explain your own code**, or if your explanation is inconsistent with what was submitted, **your submission may be called for a spot-check interview with the lecturer**. Selected pairs will be informed after submission and asked to explain their code verbally in a short session.

🚫 Submissions without a video, or with a video too short to confirm genuine collaboration and understanding, will receive **zero marks for the pair programming component**. No appeal will be entertained.

Part 2: Role Declaration in Source Code

At the very top of your `.cpp` file, before any `#include` statements, add the following comment block. Fill in the details accurately. This declaration must be consistent with what is shown in your video.

```
// =====
// SECJ1023 - Programming Technique II
// Exercise 2: Association, Aggregation & Composition
// Semester 2, 2025/2026
// =====
// Pair Information:
//   Member 1 (Name)      : _____
//   Member 1 (Matric No.) : _____
//   Member 2 (Name)      : _____
//   Member 2 (Matric No.) : _____
// -----
// Role Declaration:
//   Task 1 - Driver: _____ Navigator: _____
//   Task 2 - Driver: _____ Navigator: _____
```

```
// Task 3 - Driver: _____ Navigator: _____
// Task 4 - Driver: _____ Navigator: _____
// Task 5 - Driver: _____ Navigator: _____
// -----
// Both members confirm: the above information is accurate
// and reflects our actual pair programming session.
// =====
```

⚠ **The Role Declaration must match your video.** If the video shows Member A as Driver for Task 3 but the comment says otherwise, the video takes precedence. Inconsistencies may be raised during a spot-check.

⚠ Academic Integrity & Penalties

- You may refer to lecture notes, textbooks, and online resources. **Code that is copied or shared between pairs will be treated as plagiarism** and will result in **dismissal of the entire submission with no appeal**.
- You may use AI tools **as a reference only**; the same way you would use a textbook. **Submitting code that is substantially generated by AI without understanding is academic misconduct.** Your video and spot-check are the evidence of genuine understanding.
- Programs that cannot compile** will receive a **50% penalty on the code score**.
- Late submissions** will be penalised at **10% per 15 minutes** past the deadline.

💻 Question: Pet Clinic Management System

Write a complete C++ program that models a simple **pet clinic management system** using **composition**. The system records exactly **3 pets**. Each pet has a **name**, an **owner name**, and a **weight (kg)**. Each pet is also associated with exactly **one vaccine record**, which holds the **vaccine name**, the **dose number** administered (1, 2, or 3), and the **cost per dose (RM)**. The program calculates the total vaccination cost, determines whether a pet needs a booster dose, and displays a clinic report.

Class Design

This program uses **two classes**: `Vaccine` and `PetRecord`. `PetRecord` contains a `Vaccine` object as a data member; this is **composition**. Declare both classes exactly as shown below. All member functions must be declared as **prototypes** inside the class and defined **outside** using `::`.

Class 1: `Vaccine`

Category	Member	Access
Data Members	<code>string vaccineName</code>	private
	<code>int doseNumber</code>	private
	<code>double costPerDose</code>	private
Member Functions	<code>bool setVaccine(string, int, double)</code>	public
	<code>string getVaccineName() const</code>	public

	<code>int getDoseNumber() const</code>	public
	<code>double getCostPerDose() const</code>	public
	<code>double totalDoseCost() const</code>	public

Class 2: PetRecord

Category	Member	Access
Data Members	<code>string petNames[MAX_PETS]</code>	private
	<code>string ownerNames[MAX_PETS]</code>	private
	<code>double weights[MAX_PETS]</code>	private
	<code>Vaccine vaccines[MAX_PETS]</code>	private
Member Functions	<code>bool setDetails(int, string, string, double)</code>	public
	<code>string getPetName(int) const</code>	public
	<code>string getOwnerName(int) const</code>	public
	<code>double getWeight(int) const</code>	public
	<code>bool needsBooster(int) const</code>	public
	<code>double totalCost() const</code>	public
	<code>void displayRecord() const</code>	public

Global constant: `const int MAX_PETS = 3;` declared above both class definitions. Use this constant in all array declarations and loops.

Task 1: Class Declaration and Private Members

Declare the `Vaccine` class with private data members `vaccineName`, `doseNumber`, and `costPerDose`. Declare the `PetRecord` class with four private array data members: `petNames`, `ownerNames`, `weights`, and `vaccines` (an array of `Vaccine` objects). Declare all public member functions as prototypes inside each class.

Task 2: `setVaccine()` and `setDetails()` - Mutator Functions

Write the mutator function for the `Vaccine` class:

```
bool setVaccine(string vName, int dose, double cost)
```

This function must:

- Store `vName`, `dose`, and `cost` into the corresponding private members.

- **Validate before storing:** if `dose < 1` or `dose > 3`, **display:** "Invalid dose. Dose number must be 1, 2, or 3." and **return** `false`. If `cost <= 0`, **display:** "Invalid cost. Cost must be greater than RM0.00." and **return** `false`.
- If all inputs are valid, store `vName`, `dose`, and `cost` into the corresponding private members and **return** `true`.

Write the mutator function for the `PetRecord` class:

```
bool setDetails(int index, string pet, string owner, double weight)
```

This function must:

- Store `pet`, `owner`, and `weight` into the arrays at position `index`.
- **Validate before storing:** if `weight <= 0`, **display:** "Invalid weight. Weight must be greater than 0." and **return** `false`.
- If the input is valid, store `pet`, `owner`, and `weight` into the arrays at position `index` and **return** `true`.

 **Tip for the Navigator:** Validate **before** any array assignment. If any invalid input is detected in `setDetails()` or `setVaccine()`, the program must display the appropriate error message and terminate immediately. The program should not continue asking for the remaining information.

Task 3: Accessor Functions

Write the following accessor functions for each class. All must be declared with `const` since they do not modify any data:

For `Vaccine`:

- `string getVaccineName() const` - returns the vaccine name.
- `int getDoseNumber() const` - returns the dose number.
- `double getCostPerDose() const` - returns the cost per dose.
- `double totalDoseCost() const` - returns the total cost for all doses administered: `costPerDose * doseNumber`.

For `PetRecord`:

- `string getPetName(int index) const` returns the pet name at the given index.
- `string getOwnerName(int index) const` returns the owner name at the given index.
- `double getWeight(int index) const` returns the weight at the given index.

Task 4: `needsBooster()` and `totalCost()`

Write the following functions for the `PetRecord` class:

- `bool needsBooster(int index) const` returns `true` if the dose number of `vaccines[index]` is less than 3 (meaning the vaccination series is not complete), and `false` otherwise. Call `getDoseNumber()` on the `Vaccine` object instead of accessing the private member directly.
- `double totalCost() const` calculates and returns the total vaccination cost across all `MAX_PETS` pets. For each pet, call `totalDoseCost()` on the corresponding `Vaccine` object and sum the results.

 **Tip for the Navigator:** In `totalCost()`, call `vaccines[i].totalDoseCost()` inside the loop. Do not re-implement the calculation.

Task 5: displayRecord() and main()

`void displayRecord()` const displays a formatted clinic report showing, for each pet: the pet name, owner name, weight, vaccine name, dose number, cost per dose, total dose cost, and booster status ("Booster needed" or "Vaccination complete"). At the end, display the total vaccination cost for all pets. Call `needsBooster()` and `totalCost()` in this function.

Write a `main()` function that:

- i. Declares a `PetRecord` object named `clinic`.
- ii. Uses a `for` loop to prompt the user for each pet's name, owner name, and weight. Call `setDetails()` for each pet.
- iii. Calls `setDetails()` for each pet and checks its return value. If `setDetails()` returns `false`, terminate the program immediately using `return 0`.
- iii. Inside the same loop, prompts the user for the vaccine name, dose number, and cost per dose. Call `setVaccine()` on `clinic.vaccines[i]` directly, since `vaccines` is a private member, use a public accessor or restructure as needed; or expose the `Vaccine` object via a getter if necessary.
- iv. Calls `clinic.displayRecord()` to display the full clinic report.

 **Tip for the Navigator:** Think carefully about how `main()` can access the `vaccines` array to call `setVaccine()`. You may add a public accessor `Vaccine& getVaccine(int index)` to `PetRecord`, or call `setVaccine()` via a public method in `PetRecord`. Make sure the design follows proper encapsulation.

Important Notes

- Write the entire program in a **single source code file (.cpp)**.
- The `.cpp` file must include the **Role Declaration comment** at the top and the **Reflection answers** as comments at the bottom. See the respective sections for the required format.
- No constructors, destructors, or dynamic memory are required.
- All member functions declared in the class must be called at least once in `main()` or through other member functions.
- All function definitions must use the scope resolution operator (`::`).
- Separate the class declaration (inside the class body) from the function definitions (outside the class).

Sample Program Output

Example 1: All records entered correctly

```
Enter pet 1 name: Buddy
Enter pet 1 owner name: Ali Hassan
Enter pet 1 weight (kg): 8.5
Enter vaccine name: Rabies
Enter dose number (1-3): 2
Enter cost per dose (RM): 35.00

Enter pet 2 name: Luna
Enter pet 2 owner name: Siti Rahim
Enter pet 2 weight (kg): 4.2
Enter vaccine name: Parvo
Enter dose number (1-3): 3
```

```
Enter cost per dose (RM): 40.00
```

```
Enter pet 3 name: Max
Enter pet 3 owner name: John Tan
Enter pet 3 weight (kg): 12.0
Enter vaccine name: Distemper
Enter dose number (1-3): 1
Enter cost per dose (RM): 30.00
```

```
===== Pet Clinic Report =====
```

Pet	Owner	Wt(kg)	Vaccine	Dose	Cost/Dose	Total	Status
Buddy	Ali Hassan	8.50	Rabies	2	RM 35.00	RM 70.00	Booster needed
Luna	Siti Rahim	4.20	Parvo	3	RM 40.00	RM 120.00	Vaccination complete
Max	John Tan	12.00	Distemper	1	RM 30.00	RM 30.00	Booster needed

```
Total vaccination cost for all pets: RM 220.00
=====
```

Example 2: Invalid input

```
Enter pet 1 name: Koko
Enter pet 1 owner name: Mira Yusof
Enter pet 1 weight (kg): -3.0
Invalid weight. Weight must be greater than 0.
```

Reflection Questions

Answer the following questions **honestly and in your own words**. Write your answers as comments at the **bottom of your .cpp file**, after your `main()` function. There is no minimum or maximum word count; what matters is that your answer reflects **your own genuine experience** during this exercise.

Question 1: Describe **one error or bug** that you encountered while writing your code. What was the error message or symptom? What did you do to identify the cause, and how did you fix it?

Question 2: In this exercise, your program uses **two classes**. Explain in your own words **how the two classes are related**; is it composition, aggregation, or association? Give a specific reason based on your own code, not a general definition.

Question 3: What was the **most challenging part** of this exercise for your pair? How did you and your partner work through it together?

```
// =====
// REFLECTION
// =====
// Q1 - Error encountered:
//
//
//
// Q2 - Relationship between the two classes:
//
//
//
// Q3 - Most challenging part:
//
//
//
// =====
```

⚠️ Reflection answers that are vague, copied from another pair, or clearly written by AI (e.g. no specific error mentioned, generic definitions, no personal experience) will receive **zero marks for the pair programming component**. Write what actually happened during your session.

Assessment

This exercise carries **3%** of the final course grade. The breakdown (out of 100 points) is as follows:

Criteria	Points
Problem Solving (Code)	
a. Task 1: Class declarations, private members, and access specifiers	15
b. Task 2: setVaccine() and setDetails() mutators with input validation	15
c. Task 3: Accessor functions	10
d. Task 4: needsBooster() and totalCost()	20
e. Task 5: displayRecord() and main()	20
Pair Programming	
a. Active collaboration: Both members discuss, explain, and solve problems together throughout. Evidenced by video; the Driver explains the code out loud and the Navigator responds actively.	10
b. Role switching: Both members take turns as Driver and Navigator at least once. Evidenced by video (verbal announcement of role switch) and the Role Declaration comment in the source code.	10
Total	100

How pair programming is assessed: The lecturer will review the video recording and the Role Declaration comment in the source code. Selected pairs may be called for a short spot-check to verify understanding of their own code. Both the video and the spot-check (if applicable) contribute to the pair programming marks.

Submission

- Only one member from each pair needs to submit.
- Submit via **elearning** only. Submissions via email, Telegram, Google Drive, or any other platform will **not be accepted**.
- You must submit **THREE items**:
 - i. The source code: submit only the `.cpp` file. The file must contain the **Role Declaration comment** at the top and the **Reflection answers** as comments at the bottom.
 - ii. The video link: submit the Google Drive link via **elearning**.
 - iii. Note: The Reflection answers are embedded inside the `.cpp` file, not submitted separately.

- Deadline: **Tuesday, 26th May 2026**, 23:59 MYT (UTC+8).

? FAQs

1. **Who will be my partner?**

Your partner from Exercise 1.

2. **Can I pair up with someone from a different section?**

No. You must work with a partner from the same section.

3. **Can I do the exercise alone?**

Only if the number of students in the class is odd and you have obtained prior permission from the lecturer.

4. **Do we need to switch roles?**

Yes. Both members must experience both the Driver and Navigator roles at least once. The switch must be announced clearly in the video.

5. **Can I use AI tools such as ChatGPT or GitHub Copilot?**

You may use AI tools as a reference; for example, to understand a concept or look up syntax, the same way you would use a textbook. However, you must not use AI to generate your code for you. Your video must show that you understand and can explain every part of your code. If your explanation in the video is inconsistent with your submitted code, or if you cannot explain your code during a spot-check, you will receive zero marks for the pair programming component.

6. **What exactly must I say in the video when coding?**

While you are typing code, explain what you are doing and why. For example: "I am declaring the Vaccine array inside PetRecord because the Vaccine cannot exist without PetRecord, this makes it composition." You do not need to read every line of code, but you must explain your design decisions and the OOP concepts you are applying.

7. **What is a spot-check?**

After submission, the lecturer may select certain pairs for a short verbal session (approximately 5 to 10 minutes) where each member is asked to explain parts of their submitted code. This is to confirm that the work is genuinely understood. You will be informed by the lecturer if your pair is selected.

8. **What if we encounter an error we cannot fix?**

Document it in your Reflection (Question 1). A program that does not fully work but includes an honest reflection about the error will be assessed more favourably than a suspiciously perfect submission with no reflection.

9. **What if my partner is not contributing?**

Inform the lecturer as soon as possible, before the deadline. Do not wait until after submission.

10. **What if we do pair programming face-to-face?**

You may sit side by side and use one computer. You do not need to use Live Share or an online meeting tool. Record the session locally using OBS or any screen recorder. Both members must still speak, discuss, and switch roles visibly in the video.

11. **Will code marks and pair programming marks be assessed separately?**

Yes. Code accounts for 80 points and pair programming accounts for 20 points. Both contribute to your final 100-point score.